

PixelTagger (AnnotaVision) 项目成果与学习总结

报告状态: 基于 2026-07-17 当前分支 exp/image-rect-mcb 的源码、CMake 配置与 Git 历史整理。课程信息、成员经历、GUI 人工测试结果和截图仍需按 REPORT_INPUTS.md 与 SCREENSHOT_CHECKLIST.md 补充。

摘要

PixelTagger (可执行程序及窗口名为 AnnotaVision) 是面向图像浏览、矩形框标注和基础图像处理场景的 Windows 桌面程序。项目以 C++17、Qt Widgets 和 CMake 为基础, 通过 Model、ViewModel、View 以及负责对象装配和信号绑定的 Application 组合根划分职责。当前源码已经形成从图片导入、文件夹浏览、缩放平移、矩形标注、多类别管理、标注选择与编辑, 到 JSON 项目保存/读取、OpenCV 处理预览和 YOLO 数据集导出的可编译闭环; Qt signal/slot 用于传递用户请求、状态变化和错误反馈, Model 作为项目数据的唯一业务事实来源。2026-07-17 在本地 VS 2022 构建目录中重新构建成功, CTest 共登记 9 项测试, 其中 8 项通过, YoloExporterTests 经 CTest 调用失败, 但同一可执行文件直接运行退出码为 0, 该执行差异仍需定位, 因此不能声称测试全部通过。项目目前只支持矩形框, 尚无多边形、画笔、撤销/重做和自动保存; 图像扫描与大图加载也仍是同步、整批方式。本文严格区分可由代码和构建验证的成果、仍需人工 GUI 验收的部分与后续计划, 并为真实截图、团队资料和个人总结保留补充位置。

1. 项目概述

1.1 项目背景

图像数据进入训练、检索或质量分析流程前, 通常需要完成浏览、筛选、类别定义和空间标注。桌面工具可以直接利用本地文件系统和鼠标交互, 减少手工记录坐标的成本, 并让标注状态、项目文件和导出结果形成可复核的数据链。PixelTagger 以课程工程实践为载体, 在有限规模内探索 Qt GUI、C++ 对象设计、MVVM 分层、持久化和协作开发的组合方式。

1.2 项目目标

当前阶段目标是: 可靠导入单图或图片文件夹; 浏览并显示图片状态; 提供缩放、平移及矩形框创建、选择、移动、调整大小和删除; 管理多个类别; 保存/读取 JSON 项目; 预览并另存基础 OpenCV 处理结果; 导出 YOLO 数据集; 通过 CMake 统一构建并为核心逻辑配置 Qt Test。

完整项目的远期目标是形成更成熟的标注工作台, 包括多边形或画笔工具、撤销/重做、自动保存、更多导入导出格式、大数据量异步加载、部署与持续集成。远期目标均不是当前成果。

1.3 项目完成情况

功能模块	当前状态	实现位置	说明
工程初始化	已实现	CMakeLists.txt、CMakePresets.json	C++17, 生成 AnnotaVision, 配置 Qt Widgets、OpenCV、CTest 与 VS 预设
打开单张图片	已实现	MainWindow.cpp、ImageViewModel.cpp	文件对话框发出请求, 支持 PNG/JPG/JPEG/BMP/GIF
打开图片文件夹	已实现	ImageViewModel::importFolder	非递归按名称扫描可读图片; 遇到不可读文件则整次导入失败
上一张/下一张	已实现	MainWindow::createMenu、ImageViewModel、ProjectModel	菜单及 PgUp/PgDown、Ctrl+Left/Ctrl+Right; 首尾给出状态提示

功能模块	当前状态	实现位置	说明
状态栏反馈	已实现	ImageViewModel::loadCurrentImage、MainWindow::showStatus	显示文件名称、分辨率和当前位置/总数
异常提示	已实现	各 ViewModel、Application::bindFeedback、MainWindow::showError	业务错误经 backdoorOccurred 显示警告框
矩形图像标注	已实现，需 GUI 截图验收	ImageCanvas、AnnotationViewModel、ProjectModel	原图坐标保存；支持创建、选择、移动、四角调整和删除；无多边形/画笔
类别管理	已实现，需 GUI 截图验收	LabelViewModel、MainWindow	新增、删除、重命名、改色、切换和修改标注类别
项目保存/读取	已实现	JsonProjectRepository、ProjectViewModel	使用 Qt JSON API；图片路径按项目文件位置解析，不使用 nlohmann-json
YOLO 导出	已实现但测试执行有待复核	YoloExporter、ProjectViewModel::exportYolo	输出图片、标签、classes.txt、data.yaml；CTest 中该测试目标本轮异常
OpenCV 图像处理	已实现	ImageProcessor、ProcessViewModel、处理停靠面板	灰度、二值、三类滤波、Canny、亮度、对比度预览和另存
自动化测试	部分通过	tests/、CMakeLists.txt	本轮 9 项中 8 项通过；不提供虚构覆盖率

2. 需求分析

2.1 功能需求

已落地需求包括图片导入与导航、状态和错误反馈、画布适应显示、缩放和平移、矩形标注、多类别、标注编辑、项目持久化、基础处理预览、YOLO 导出。其共同约束是：界面只产生请求和展示结果，业务状态由 ViewModel/Model 管理。

后续扩展需求包括多边形和画笔标注、撤销/重做、自动保存、最近项目、批量任务、更多格式和异步加载。JSON 与 YOLO 已存在，因此不再将它们误列为“尚未实现”；XML 及通用 JSON 标注格式转换仍未发现实现。

2.2 非功能需求

- 可维护性：目录按 model、viewmodel、view、repository、processor、exporter 和 app 分层，类职责较明确。
- 可扩展性：ViewModelChange 枚举和展示 DTO 降低大型数据直接穿过 signal 的频率；新增变化类型仍需同步维护绑定分支。
- 界面响应性：当前文件夹扫描、图片解码、OpenCV 处理和导出均在 GUI 调用链中同步执行，大图或大目录可能阻塞界面。
- 错误处理：Result<T> 承载业务失败，ViewModel 转换为 errorOccurred；部分边界只显示状态栏而非弹窗。
- 跨平台潜力：Qt/CMake/OpenCV 均具跨平台能力，但预设、部署命令和本轮验证以 Windows/MSVC 为主。
- 模块解耦：Model 不继承 QWidget；View 不持有 Model/ViewModel，绑定集中在 Application。
- 编译部署：CMake 提供多套 VS 预设并部署 Windows 平台插件；依赖仍需要本地 Qt/OpenCV/vcpkg 环境。

2.3 使用流程

1. 构建并启动 AnnotaVision.exe，检查空画布和“就绪”状态。

2. 使用“文件 → 打开图片”选择 PNG、JPG、JPEG、BMP 或 GIF。
3. 或使用“打开图片文件夹”加载目录中按名称排序的图片。
4. 使用“浏览”菜单、PgUp/PgDown 或 Ctrl+Left/Ctrl+Right 切换图片。
5. 从状态栏查看文件名、分辨率和“当前位置/总数”。
6. 在画布空白处左键拖拽创建矩形框；选择框后可移动、调整、改类别或删除。
7. 可保存 JSON 项目、预览图像处理结果或选择空目录导出 YOLO 数据集。
8. 遇到空目录、无效图片或业务错误时查看弹窗；首尾导航提示显示在状态栏。

3. 系统总体设计

3.1 技术选型

技术	项目中的用途	选择原因	当前使用状态
C++17	业务模型、算法封装和应用组合	类型系统、RAII, 与 Qt/CMake 配合成熟	已使用
Qt Widgets	窗口、菜单、对话框、绘制、JSON、测试、信号槽	适合传统桌面 GUI	已使用, CMake 兼容 Qt 5/6; 本地缓存为 Qt 6
CMake 3.20+	目标、依赖、测试和 VS 工程生成	跨 IDE、可复现配置	已使用; 本轮命令为 CMake 4.2.0
Visual Studio/MSVC	Windows 编译和调试	与 Qt Windows kit、CMake 生成器配合	VS 2022 本轮构建成功
vcpkg	为 CMake 提供 Qt/OpenCV 工具链和运行时部署	统一第三方依赖发现	已实际接入缓存与 VS 预设, 不只是计划
OpenCV core/imgproc	基础图像处理	提供成熟矩阵与滤波算法	已正式链接并有测试
Qt JSON API	项目序列化	无需增加额外 JSON 依赖	已使用; 未发现 nlohmann-json
Qt Test/CTest	Model、ViewModel、Canvas、处理和导出测试	与 Qt 对象和信号测试集成	已配置; 本轮 8/9 通过

3.2 系统架构

当前实现是“MVVM + Application 组合根 + Repository/Processor/Exporter”。与早期提交 6081e02 的 Command 尝试不同, 当前工作树中不存在 ICommandBase、TupleCommand、std::any 或 std::any_cast。View 通过强类型 Qt signal 发出请求, Application 负责连接到 ViewModel 的公有槽/方法; 因此不能按旧设计声称 View 从 ViewModel 获取 Command, 也不存在 imageChanged 信号, 图片大数据变化统一使用 changed(ViewModelChange::CurrentImage)。

flowchart LR

```

U[用户操作] --> V[View: MainWindow / ImageCanvas]
V -- 请求 signals --> A[Application 组合根]
A -- connect --> VM[Image/Annotation/Label/Project/Process ViewModel]
VM --> M[ProjectModel]
VM --> S[Repository / Processor / Exporter]
M --> VM
VM -- changed / statusChanged / errorOccurred --> A
A --> V
V --> UI[画布、工具栏、状态栏、弹窗]

```

Qt signal 与旧 Command 的职责不同: signal 适合对象间异步风格通知和一对多连接; Command 通常将“可执行操作”包装成对象, 便于统一启用状态、撤销或队列化。当前强类型 signal 避免了 `std::any` 参数错误只能运行时发现及 `std::bad_any_cast` 风险, 但尚未提供通用撤销/重做命令历史。如果未来重新引入 Command, 宜采用模板化或明确参数类型, 而不是无约束 `std::any`。

3.3 项目目录结构

```
PixelTagger/
|-- CMakeLists.txt
|-- CMakePresets.json
|-- README.md
|-- docs/
|   |-- mvvm.md
|   |-- *-team-handoff.md
|   `-- report/
|-- src/
|   |-- app/           # Application 对象装配与信号绑定
|   |-- common/        # ID、Result、变化枚举和展示 DTO
|   |-- model/         # 项目、图片、类别、标注实体
|   |-- viewmodel/     # 图片、标注、类别、项目和处理业务
|   |-- view/          # MainWindow、ImageCanvas、坐标映射
|   |-- repository/    # JSON 项目持久化
|   |-- processor/     # Qt/OpenCV 转换与图像算法
|   |-- exporter/      # YOLO 数据集导出
|   `-- main.cpp
`-- tests/             # Qt Test 自动化测试
```

src/main.cpp 只创建 QApplication 和 Application; Application 保证共享 ProjectModel 只有一个实例。common/presentation 的 DTO 防止 View 直接依赖可写 Model。docs/mvvm.md 记录当前调用边界, 三份 handoff 文档记录类别编辑、图像处理 and YOLO 集成背景。

3.4 核心数据流

打开单图时, MainWindow 选择路径并发出 `importImageRequested`; Application 将其连接到 `ImageViewModel::loadImage`; ViewModel 用 `QImageReader` 读取元数据, 调用 `ProjectModel::replaceImages`, 再解码当前图并发出变化和状态信号。

加载文件夹时, `ImageViewModel::importFolder` 使用扩展名过滤并按名称扫描, 构建完整 `QVector<ImageModel>` 后一次性替换 Model。上一张/下一张由 ProjectModel 改变索引, 边界失败时只发布状态文本。所有 ViewModel 错误最终由 `Application::bindFeedback` 连接至 `MainWindow::showError`。

```
sequenceDiagram
    participant User as 用户
    participant View as MainWindow
    participant App as Application
    participant VM as ImageViewModel
    participant Model as ProjectModel
    participant Canvas as ImageCanvas
    User->>View: 选择“打开图片”
    View-->>App: importImageRequested(path)
    App->>VM: loadImage(path)
```

```

VM->>VM: QImageReader 校验并读取图片
VM->>Model: replaceImages(images)
VM-->>App: changed(CurrentImage)
App->>VM: currentQImage()
App->>Canvas: setImage(image)
VM-->>App: statusChanged(message)
App->>View: showStatus(message)
Canvas->>Canvas: update() 后 paintEvent()

```

4. 核心模块实现

4.1 主窗口模块

MainWindow 创建“文件、浏览、视图”菜单、标注工具栏和图像处理停靠面板。文件对话框负责收集图片、文件夹、JSON 项目、导出目录及处理结果路径；窗口不直接修改 ProjectModel。图片打开使用 QKeySequence::Open，保存项目使用 QKeySequence::Save，浏览使用 PgUp/PgDown 与 Ctrl+ 方向键，删除标注使用 Delete，缩放和适应窗口也有快捷键。状态反馈进入 QStatusBar，错误通过 QMessageBox::warning 展示。

其局限是菜单和处理面板均在一个实现文件中，规模继续扩大后可拆成专用 widget/action factory；此外 GUI 请求由 signals 暴露，依赖 Application 完整绑定，缺少绑定会静默失效。

4.2 图像显示模块

ImageCanvas::paintEvent 先绘制深色背景；无图时居中显示“打开一张图片开始”。有图时根据 Qt::KeepAspectRatio 计算视口，结合缩放因子和偏移居中绘制，设置 QPainter::SmoothPixmapTransform 实现平滑显示。setImage 清理交互状态、重置缩放和平移、更新坐标映射并调用 update()；setAnnotations 同样触发重绘。

CoordinateMapper 在窗口坐标与原图坐标间转换，因此标注不会因窗口缩放改变业务坐标。Canvas 还支持滚轮缩放、中键平移、命中选择、矩形创建、移动和四角缩放。当前只提供矩形，且大图作为完整 QImage 保存在内存中，没有瓦片化或分级缓存。

4.3 图片业务模块

ImageViewModel 集中处理单图导入、文件夹扫描、当前图解码和导航。支持后缀为 PNG、JPG、JPEG、BMP、GIF；GIF 通过普通 QImage 加载路径处理，代码中没有动画帧播放逻辑，因此只应视为静态显示。readImage 保存绝对路径、文件名、相对路径和宽高，导入时初始化 modified=false。状态文本格式为“文件名 宽 x 高 当前位置/总数”。

首尾导航不改变索引，并发布“已经是第一/最后一张图片”。空文件夹、目录不存在、图片不可读或当前图解码失败则发出 errorOccurred。目录扫描非递归且同步；任一候选图片不可读会终止整批导入，这在大量混合文件目录中不够宽容。

4.4 数据模型模块

ImageModel 保存 id、文件绝对/相对路径、文件名、宽高、修改状态和标注列表。AnnotationModel 保存标注 ID、类别 ID 和原图坐标 QRectF；LabelModel 保存类别 ID、名称和颜色。

ProjectModel 保存图片列表、类别列表、当前索引、项目修改状态及下一组实体 ID。它提供受控的替换项目、导航、增加/删除/更新标注、类别增删改查、占用检查和 markSaved，避免 View 获得可写数据指针。当前项目没有单独的项目名称、创建时间、作者或版本字段，JSON schema 演进能力需要继续检查和设计。

4.5 请求绑定与 Command 架构演进

当前调用边界是：View 发出强类型请求 signal，Application 将请求连接到对应 ViewModel；ViewModel 修改 Model，再以 changed、statusChanged、errorOccurred 等信号反向通知。这降低了 View 与具体业务实现的耦

合，并保留统一接入快捷键的入口。

仓库历史中曾出现 “add ICommandBase class for future function addition” 提交，但当前文件树已无 ICommandBase 和 TupleCommand。因此，旧 Command 方案的 “std::any 携带 std::tuple、解包后调用 ViewModel” 只能作为历史设计讨论，不能写为现状。若以后为撤销/重做重新设计 Command，应让每个命令具有明确参数、执行/回退语义和可测试的失败结果；这比 std::any_cast 更能在编译期发现类型不匹配。

4.6 标注、类别、处理与导出模块

AnnotationViewModel 将拖拽框归一化并裁剪到图片边界，拒绝小于 3×3 原图像素的框；它维护当前类别与选中标注 ID，并通过 AnnotationRenderData 向 Canvas 提供颜色、名称和选中状态。LabelViewModel 处理类别新增、删除、重命名、改色和当前类别切换；正在使用的类别由 Model 保护。

ProcessViewModel 使用 ImageConverter 在 QImage 与 cv::Mat 之间转换，调用 ImageProcessor 生成非破坏性预览，支持恢复原图与另存结果。预览不直接回写项目图片或标注坐标。JsonProjectRepository 负责项目文件，YoloExporter 验证图片、类别与矩形后在暂存目录生成数据集，并拒绝非空导出目录和重名输出。YOLO 当前将同一 images 同时写作 train/val 路径，尚未实现数据集划分策略。

5. 项目成果展示

下列前三张为报告必需图片引用，但文件当前尚不存在，均已在 SCREENSHOT_CHECKLIST.md 标为“待截图”。导出最终 PDF 前必须用真实运行截图补齐。

5.1 程序启动界面

程序启动后的主界面

图 5-1 程序启动后的主界面

预期画面应同时显示 AnnotaVision 标题、文件/浏览/视图菜单、标注工具栏、图像处理面板、空画布提示和“就绪”状态栏。该截图用于验证界面组成，不可由旧版 docs/images/main-window.png 直接替代，除非人工确认其与当前提交一致并重新按清单命名。

5.2 打开单张图片

成功打开单张图片

图 5-2 成功打开单张图片

真实截图应证明图片在画布中按比例居中显示，并保留状态栏的文件名、分辨率与 1/1 位置。图片内容应使用可公开的测试素材，避免暴露个人目录。

5.3 文件夹浏览和图片切换

加载图片文件夹并切换图片

图 5-3 加载图片文件夹并切换图片

截图应在加载多图目录并切换后获取，使状态栏出现非 1/1 的当前位置/总数。仅凭静态截图很难证明快捷键过程，提交时可在图注或演示记录中补充实际按键与前后位置变化。

5.4 错误与边界处理

当前未引用 04-error-handling.png，因为尚未取得真实证据。建议分别验证空目录警告、不可读图片警告、第一张执行上一张和最后一张执行下一张；弹窗与状态栏属于两类反馈，不应混写为同一行为。

5.5 协作与开发工具

可验证的协作证据是本地 Git 历史、远程 origin 和本轮 Codex 对仓库文档的整理。仓库元数据不能证明存在 Pull Request、Issue、项目看板或代码审查记录，因此正文不展示这些未验证工具。Visual Studio/CMake 构建证据和 Codex 关键对话待人工截图。

6. 构建、运行与测试

6.1 开发环境

项目	配置
操作系统	Windows; 具体版本【待填写】
编译器	MSVC; 具体工具集版本【待填写】
Visual Studio	当前缓存为 Visual Studio 2022 Community, 课程提交环境请复核
CMake	本轮命令输出 4.2.0; 项目最低要求 3.20
Qt	当前 CMake 缓存找到 vcpkg 的 Qt 6; README 记录已验证 6.11.1, 最终版本请本机复核
C++ 标准	C++17
OpenCV	已由 vcpkg/CMake 接入 core、imgproc; README 记录已验证 4.12.0
vcpkg	x64-windows 工具链已实际使用; 私人绝对路径不写入报告

6.2 构建过程

推荐按仓库预设执行：

```
cmake --preset vs2022-x64 -DCMAKE_PREFIX_PATH=<QtPrefix>
cmake --build --preset vs2022-x64 --config Debug
ctest --test-dir build/vs2022-x64 -C Debug --output-on-failure
```

也可在已配置目录中执行：

```
cmake --build build/vs2022-x64 --config Debug
```

2026-07-17 本轮构建成功生成 Debug AnnotaVision.exe，并部署 Qt Windows 平台插件及相关运行库。该结论只覆盖当前机器和 Debug 配置，不等同于其他成员环境或 Release 部署均已验证。

6.3 测试方案与本轮结果

GUI 验收表中的“实际结果”必须由成员运行并填写：

编号	测试内容	操作步骤	预期结果	实际结果	证据
T01	程序启动	启动程序	显示空画布和就绪状态	【待填写】	图 5-1
T02	打开有效图片	选择 PNG/JPG	图片正常显示	【待填写】	图 5-2
T03	加载图片文件夹	选择含多图文件夹	加载第一张图片	【待填写】	图 5-3
T04	下一张	执行下一张命令	索引增加	【待填写】	图 5-3/待补
T05	第一张边界	第一张执行上一张	状态栏给出边界提示	【待填写】	图 5-4
T06	最后一张边界	最后一张执行下一张	状态栏给出边界提示	【待填写】	图 5-4
T07	空文件夹	选择无图片文件夹	弹出错误提示	【待填写】	图 5-4
T08	无效图片	选择无法解析的候选文件	弹出错误提示	【待填写】	图 5-4
T09	窗口缩放	改变窗口大小	图片等比例重绘	【待填写】	待补截图

自动化测试方面，CMakeLists.txt 登记 9 个 Qt Test 目标，覆盖类别 Model/ViewModel、标注编辑、主窗口删除动作、Canvas 缩放与编辑、OpenCV 处理、处理 ViewModel、YOLO 导出和项目导出。2026-07-17 本轮 CTest 结果为 8/9 通过，失败项为 YoloExporterTests；随后直接运行同一 Debug 测试可执行文件得到退出码 0 且无控制台输出。该差异可能与 CTest 启动环境或测试输出方式有关，尚无充分证据确定根因，故记录为“待定位”，不宣称全部通过，也不虚构覆盖率。

7. 团队协作与项目管理

7.1 团队分工

成员	主要角色	负责模块	主要产出	协作对象
【成员 1】	【待填写】	【待填写】	【待填写】	【待填写】
【成员 2】	【待填写】	【待填写】	【待填写】	【待填写】
【成员 3】	【待填写】	【待填写】	【待填写】	【待填写】

提交者标识不能直接推断真实姓名，上表必须由团队确认。

7.2 Git 协作情况

git rev-list --count --all 显示全引用共有 23 个可见提交，当前 HEAD 也包含 23 个，main 包含 3 个。git shortlog -sn --all 显示四个作者标识：kkkkh-kh 10 次、serein6174 8 次、Ling Ni 3 次、NeEnZo 2 次。主题从工程初始化、MVVM/Application 重构、矩形标注、多类别，扩展到标注编辑、OpenCV、YOLO、UI 集成与报告整理。

当前分支为 exp/image-rect-mcb，本地/远端还可见 main；历史中存在一次普通 merge commit。仅凭本地 Git 对象无法确认是否使用 Pull Request、Issue 或正式代码审查，均标记为【待填写/待提供平台证据】。

Git 提交数量只能反映仓库中可见的提交记录，不能完全代表各成员的实际工作量。

7.3 模块集成方式

团队可围绕稳定接口并行：Model 约定实体 ID、原图矩形与 dirty 语义；ViewModel 暴露业务方法和强类型信号；Application 统一连接；View 只处理绘制和输入；CMake 负责把新增源文件、库和测试接入目标。实际由谁负责哪一层、合并中发生过哪些冲突仍需成员填写，不能从提交主题机械推断完整分工。

8. 智能体使用说明

8.1 智能体基本配置

本报告文件由当前 Codex 会话在仓库根目录内检索、生成和校验，说明项目本轮确实使用了智能体；更早的代码是否由智能体生成须由成员确认。

配置项	实际配置
智能体名称	Codex
使用方式	当前为仓库工作区会话；CLI、IDE 或其他入口【待填写】
使用模型	【待填写】
工作目录	PixelTagger 仓库根目录
权限模式	工作区写权限；对外部范围的具体策略【待填写】
网络访问	本轮报告整理未使用网络；平台配置【待填写】
自动执行命令	本轮经会话执行只读审计、构建、测试与文档写入

配置项	实际配置
人工审查机制	代码事实由命令核对；GUI、成员经历和最终提交仍需人工复核

报告不记录 API Key、Token、密码、私有服务器地址或个人绝对路径；如后续引用配置，敏感值统一替换为 ***REDACTED***。

8.2 MCP Server 设置

仓库中未发现 AGENTS.md、项目 .codex/ 配置、MCP Server 配置或可证明实际调用 MCP 的记录；.agents/ 当前未发现可用规则内容。因此：本项目当前未配置可由仓库验证的 MCP Server，报告不虚构相关设置过程。若成员在仓库外配置过 MCP，应只在提供脱敏证据后补写。

8.3 规则文档与技能文档

仓库存在 docs/mvvm.md，明确约束 View 不直接修改 Model、ViewModel 负责业务和展示数据、Model 不依赖 QWidget、大数据变化经 changed(ViewModelChange) 通知并由 Application 拉取 getter。还存在类别编辑、图像处理 and YOLO 的交接文档。未发现项目级 AGENTS.md、技能文档或单独编码规范；CMakeLists.txt 的 /utf-8 是可验证的编译约束。建议后续把构建检查、修改范围、禁止虚构功能和敏感信息规则整理成项目级贡献指南。

8.4 关键人机对话记录

轮次	人类任务	智能体执行内容	人工检查	发现的问题	最终处理
1	整理最终实验报告材料	审计源码、CMake、文档与 Git，创建报告及辅助文件	【待填写】	任务描述基于早期 Command/浏览功能，当前代码已扩展	报告按当前分支事实重写
2	验证构建与测试	重建 Debug 并运行 9 项 CTest	【待填写】	YOLO 测试在 CTest 下失败、直接运行退出 0	如实记录差异，待定位
3	补截图并导出 PDF	【待人工执行】	【待填写】	截图尚缺，最终 PDF 尚不能作为完整成果	按清单补图后运行脚本

此表仅记录本轮已发生内容，不复制完整对话；人工审核与修正过程由团队补充。

8.5 团队成员与智能体的分工机制

结合本项目，人的职责是确认功能范围和 MVVM 边界、决定 JSON/YOLO 兼容策略、实际操作 GUI、判断交互质量、提供成员经历并完成最终合并。智能体适合批量检索类和信号、对照 CMake/Git、生成初版文档、运行重复性检查。涉及 UI 手感、截图真实性、测试异常根因和个人反思时，必须由成员验证，不能仅凭静态分析定稿。

8.6 实际效果

本轮智能体提高了跨文件事实汇总、Git 统计、构建测试和报告骨架整理效率，并识别出“当前已无 Command 层”“OpenCV/YOLO/类别功能已经接入”“CTest 并非全通过”等与任务初始假设不一致之处。代码模块是否由此前智能体辅助生成、是否发生编译失败或过度修改，仓库没有充分证据，保留【待填写】。本轮智能体只新增报告材料，没有改动产品源代码。

8.7 当前问题与风险

- 智能体可能按旧描述误判现有功能，必须以当前工作树为准。
- Qt/C++ API 和 moc/CMake 接入容易产生“局部看似正确但无法编译”的代码。
- 若直接让 View 修改 Model，会破坏 docs/mvvm.md 的边界。
- 建议代码可能引用未安装依赖，例如当前项目并未使用 nlohmann-json。
- 自动化测试目标存在不代表全部通过，更不等于覆盖率达标。
- 工作区可能包含成员未提交修改，写入前必须检查状态并避免覆盖。
- 静态分析无法可靠判断 GUI 实际尺寸、操作手感和截图效果。
- 最终仍需要人工编译、运行、截图、复核导出数据和验收 PDF。

9. 项目实际效果与问题分析

9.1 当前成果

可验证成果包括：C++17/Qt/CMake 工程可重新构建；单图和目录导入、导航与状态反馈有完整代码路径；Canvas 实现等比显示、缩放平移和矩形交互；Model/ViewModel 支持多类别、标注编辑和修改状态；项目可用 Qt JSON 保存读取；OpenCV 处理和 YOLO 导出已接入 UI，并有对应测试目标。GUI 场景和 YOLO CTest 差异仍需人工证据补强。

9.2 技术优点

目录职责清晰，Model 不依赖界面，ViewModel 集中业务状态，Application 统一请求绑定，Qt signal 完成反向通知。图片绘制、坐标映射与项目业务分离，标注坚持原图坐标；展示 DTO 避免 View 获得可写 Model；Repository、Processor 和 Exporter 形成清晰的外围能力边界。

9.3 当前不足

- 标注类型只有矩形，没有多边形、画笔或关键点。
- 缺少撤销/重做、自动保存和最近项目；modified 状态尚未形成完整关闭提醒流程。
- 文件夹一次性同步扫描和逐图读元数据，大目录可能阻塞；失败策略会因单个坏文件中止全部导入。
- 大图完整解码并复制到处理 ViewModel，内存占用和响应时间有风险。
- GIF 没有动画帧模型，只按普通静态图使用。
- YOLO 导出的 train/val 指向同一目录，缺少划分；CTest 启动异常未定位。
- JSON 持久化依赖 Qt API，尚未发现 schema 版本迁移与兼容测试说明；nlohmann-json 未接入。
- 测试主要覆盖组件，不等于端到端 GUI 验收，也没有覆盖率数据。
- 构建和部署依赖本地 Qt/OpenCV/vcpkg 与 Windows 工具链。

9.4 后续改进计划

P0: 巩固最小标注闭环 当前矩形、类别、保存/读取和修改状态已有基础。P0 应优先补充关闭前未保存提示、项目 schema 版本、重新加载回归测试、YOLO CTest 异常定位，并以真实 GUI 测试确认创建—编辑—保存—重开闭环；再评估多边形等新标注实体。

P1: 工程可用性 引入强类型撤销/重做命令、自动保存、最近项目、操作快捷键说明、异步目录扫描、坏图跳过策略和更细粒度错误信息。缩放和平移已实现，P1 的重点是补充体验验证与边界优化，而非把它们重复列为新功能。

P2: 格式与处理能力 扩展通用 JSON/XML 标注导出、YOLO 数据集划分、批处理和更多 OpenCV 流程；只有在确有跨库需求时再评估 nlohmann-json。以上均为后续计划，不是当前成果。

10. 学习回顾与团队总结

10.1 知识掌握

从当前代码可以归纳的知识点包括 C++ 对象组合与值语义、Qt 事件和信号槽、MVVM 调用边界、CMake 目标与依赖组织、Qt JSON 数据模型、原图/显示坐标转换以及 Result<T> 错误传播。旧 Command 尝试与当前 signal 方案的差异也提供了架构演进案例。成员各自掌握程度和具体实例【待本人填写】。

10.2 技能提升

项目覆盖 Qt GUI 绘制、文件对话框、CMake/vcpkg 配置、MSVC 构建、Qt Test、Git 分支历史、接口设计和智能体辅助审计。实际由谁完成何种调试、采用何种定位方法及智能体建议如何被审核，需结合成员记录填写，不能由提交数代替。

10.3 团队协作

代码结构适合按 Model/ViewModel、View/Canvas、处理、导出和构建测试拆分，并通过实体 ID、signals 和 CMake 目标约定集成。真实的提交规范、合并冲突、进度同步和联调验收案例均为【待填写】；最终总结应选取实际事件，而非套用通用协作表述。

11. 个人总结

以下仅提供不同关注点的写作提示，不代写个人经历。每位成员须结合真实工作独立完成，建议“深度反思”不少于 500 字。

11.1 成员一：【姓名】

承担角色 【待本人填写：角色及与其他模块的接口】

完成工作 【待本人填写：文件、功能、测试或文档证据】

主要挑战 【待本人填写：重点回顾架构边界或数据模型问题】

应对策略 【待本人填写：实际尝试、失败方案和最终选择】

能力成长 【待本人填写：C++/MVVM/接口设计方面的具体变化】

深度反思 【待本人独立撰写，建议不少于 500 字】

11.2 成员二：【姓名】

承担角色 【待本人填写：角色及交互或可视化职责】

完成工作 【待本人填写：界面、Canvas、坐标或用户反馈证据】

主要挑战 【待本人填写：重点回顾 Qt 事件、绘制或联调问题】

应对策略 【待本人填写：调试工具、测试场景和协作沟通】

能力成长 【待本人填写：Qt GUI 与用户体验判断方面的收获】

深度反思 【待本人独立撰写，建议不少于 500 字，不复制成员一】

11.3 成员三：【姓名】

承担角色 【待本人填写：角色及构建、算法、导出或质量职责】

完成工作 【待本人填写：CMake、OpenCV、YOLO、测试或集成证据】

主要挑战 【待本人填写：重点回顾依赖、测试一致性或交付问题】

应对策略 【待本人填写：环境复现、错误定位和风险控制】

能力成长 【待本人填写：工程化、测试或 Git 协作方面的收获】

深度反思 【待本人独立撰写，建议不少于 500 字，不复制前两位】

12. 课程改进建议

1. 现有问题：Qt 与 MSVC/vcpkg 组合的首次配置成本高。改进措施：课程早期提供可编译的 Qt/CMake 最小模板及环境诊断脚本。预期效果：把时间从环境猜错转向功能和架构实践。
2. 现有问题：学生容易把 MVVM 理解为目录命名。改进措施：提供含 View 请求、组合根绑定、ViewModel 更新 Model 和 signal 反向通知的可运行案例，并比较 MVC/Command。预期效果：以调用边界而非概念口号评价架构。
3. 现有问题：分支合并后才发现接口不一致。改进措施：设置 Model、ViewModel、UI 接入三个阶段性代码审查点，并要求接口变更记录。预期效果：降低集中联调风险。
4. 现有问题：只练习 commit，缺少 PR 和审查证据。改进措施：安排一次必须经过分支、Pull Request、评审意见和修正提交的协作任务。预期效果：形成可追溯的团队质量过程。
5. 现有问题：GUI 项目容易“能编译但不能交付”。改进措施：增加 Qt 插件/运行库部署、依赖管理、Debug/Release 和干净机器验证教学。预期效果：使构建成功与可运行交付得到区分。
6. 现有问题：智能体使用记录可能只保留结果，缺少审核过程。改进措施：统一记录目标、改动文件、编译结果、人工检查、二次修正和脱敏要求。预期效果：能够评价辅助效率和人的判断责任。
7. 现有问题：智能体可能虚构功能或测试。改进措施：提供基于源码位置、命令输出、截图和敏感信息检查的审核清单。预期效果：让报告结论可验证。
8. 现有问题：期末才集中发现范围失控。改进措施：安排中期演示，要求展示最小闭环、未完成清单和前三项风险。预期效果：尽早调整目标。
9. 现有问题：只看功能数量会忽视结构和协作。改进措施：评分同时覆盖可运行效果、代码质量、自动化测试、Git 过程和个人反思。预期效果：鼓励可维护成果。
10. 现有问题：截图口径不统一。改进措施：提前规定窗口尺寸、必需场景、状态栏上下文、图注和隐私遮挡。预期效果：提升报告证据的可比性与可信度。

13. 总结与展望

PixelTagger 已从早期图片浏览和 Command 设想演进为可构建的 MVVM 桌面应用：共享 Model 承载图片、类别和标注，多个 ViewModel 组织业务，Application 统一连接 View 请求与状态反馈，并扩展了 JSON、OpenCV 和 YOLO 能力。其价值不只在功能数量，还在于形成原图坐标、受控 Model 接口、展示 DTO 和外围服务分层等工程边界。

本轮实践也暴露出同步加载、大图内存、矩形类型单一、缺少撤销/自动保存、YOLO 测试执行不一致和 GUI 证据不足等问题。团队协作与智能体可以提高检索、初步实现和文档整理效率，但真实交互、测试异常、个人贡献和最终质量仍需人负责。下一阶段应先补齐截图与端到端验收、定位失败测试并巩固保存闭环，再扩展标注类型、异步能力和导出策略。

附录 A：构建与运行命令

配置 (*Qt* 不在默认路径时传入前缀)

```
cmake --preset vs2022-x64 -DCMAKE_PREFIX_PATH=<QtPrefix>
```

构建 *Debug*

```
cmake --build --preset vs2022-x64 --config Debug
```

测试

```
ctest --test-dir build/vs2022-x64 -C Debug --output-on-failure
```

运行 (路径依生成目录而定)

```
.\build\vs2022-x64\Debug\AnnotaVision.exe
```

如需补充 *Qt* 运行库

```
windeployqt .\build\vs2022-x64\Debug\AnnotaVision.exe
```

PDF 导出:

```
.\docs\report\export_report.ps1 -CjkFont "Microsoft YaHei"
```

附录 B：测试记录

GUI 测试记录见 6.3 与 REPORT_INPUTS.md; 证据文件见 SCREENSHOT_CHECKLIST.md。自动化测试本轮结果为 8/9, 失败差异已在 6.3 记录, 修复或确认前不得改写为全部通过。

附录 C：智能体关键交互记录

筛选记录见 8.4。本轮原始需求摘要为: 基于真实仓库生成中文最终实验报告, 严格区分当前成果、部分集成与未来计划, 创建截图/输入/导出辅助材料并完成验证。后续对话截图须脱敏。

附录 D：团队贡献说明

可见 Git 作者标识及提交数见 7.2。作者标识到真实成员姓名、负责模块和线下工作量的对应关系均为【待团队确认】; 最终应由成员结合实际分工、评审、联调和文档贡献共同说明, 不能仅按提交数量分配贡献。